

TRABAJO FINAL

Selección bioinspirada de una cartera de validación para Gatling AI Performance Copilot

Algoritmos genéticos aplicados a la priorización experimental de propuestas upgrade

Asignatura	Profesor
15 Sistemas Bioinspirados	Ricardo Contreras A.

Autores

Luis Araya · Rodrigo González · Hernán Medina

Magíster en Inteligencia Artificial · Universidad Adolfo Ibáñez

Agosto de 2026

Resumen

Gatling AI Performance Copilot analiza evidencia histórica de pruebas de rendimiento y genera recomendaciones review, maintain o upgrade. En la ejecución estudiada, 275 propuestas upgrade quedaron pendientes de una nueva ejecución Gatling. Validarlas todas simultáneamente no es viable cuando el tiempo, la infraestructura y la capacidad de análisis son limitados. Este trabajo formula esa necesidad como un problema de optimización combinatoria y desarrolla un algoritmo genético para seleccionar una cartera de validación de 20 casos, maximizando diversidad, cobertura y calidad histórica, bajo restricciones duras de seguridad.

Después de eliminar duplicados se obtuvieron 113 candidatos factibles. Se utilizaron 80 individuos, 120 generaciones, cruzamiento uniforme, mutación binaria, selección por torneo, elitismo y reparación del presupuesto. En diez ejecuciones independientes, el mejor fitness fue 0,99049, el promedio 0,98440 y la desviación estándar 0,00298. El algoritmo superó al promedio de 100 carteras aleatorias (0,77711) y a una heurística greedy basada en calidad individual (0,51053). La cartera final cubrió 20 builds, 4 pilares, 12 componentes, 7 cuadrantes y 20 configuraciones diferentes, sin concentración por build y cumpliendo Estado=Success, Performance=1, errorCount=0 y p95 menor o igual a 1.500 ms.

Conclusión principal: el algoritmo genético prioriza una muestra experimental diversa y segura según la evidencia histórica, pero no valida por sí mismo los upgrades. La aprobación humana y nuevas ejecuciones Gatling siguen siendo obligatorias.

Palabras clave: algoritmos genéticos, pruebas de rendimiento, Gatling, optimización combinatoria, selección de cartera, validación experimental.

Contenido

- 1. Introducción
- 2. Presentación técnica
- 3. Descripción formal del problema
- 4. Metodología y diseño del algoritmo
- 5. Implementación
- 6. Resultados
- 7. Discusión
- 8. Conclusiones y proyecciones
- 9. Referencias
- Anexo A. Cartera seleccionada
- Anexo B. Correspondencia con la rúbrica

1. Introducción

1.1 Contexto

Las pruebas de rendimiento generan configuraciones, métricas, assertions, registros e históricos que deben consolidarse antes de decidir la siguiente acción. El Capstone Gatling AI Performance Copilot automatiza ese análisis mediante reglas expertas, modelos explicables, comparación histórica y artefactos auditables. La solución conserva la intervención del especialista y no modifica automáticamente una prueba.

La versión analizada implementa cuatro capas: clasificación de aplicabilidad; decisión review, maintain o upgrade; propuesta controlada de parámetros y cuadrante; y evaluación offline con contrato de validación online. Una recomendación upgrade permanece en pending_new_execution hasta que una nueva ejecución Gatling confirme estado exitoso, cero errores y ausencia de regresión.

1.2 Problema de investigación

La evaluación completa produjo 275 propuestas upgrade pendientes. La restricción práctica no consiste en generar nuevas recomendaciones, capacidad que ya existe, sino en determinar cuáles deben validarse primero bajo un presupuesto limitado. Una selección manual o puramente greedy puede concentrarse en unos pocos pilares, componentes o configuraciones y producir evidencia insuficientemente representativa.

1.3 Objetivo general

Desarrollar y evaluar un algoritmo genético que seleccione una cartera diversa y segura de propuestas upgrade para priorizar una campaña de validación Gatling con presupuesto limitado.

1.4 Objetivos específicos

1. Integrar las recomendaciones estratificadas del Capstone con el histórico corporativo por Build_Id.
2. Definir una representación cromosómica y una función de fitness multiobjetivo.
3. Aplicar restricciones duras que excluyan evidencia fallida o incompatible con un upgrade.
4. Comparar el algoritmo genético contra selección aleatoria y una heurística greedy.
5. Evaluar convergencia, estabilidad multisentilla, diversidad y trazabilidad de la cartera final.

1.5 Alcance

El trabajo prioriza casos para experimentación. No predice el resultado futuro de una ejecución, no cambia automáticamente el cuadrante y no demuestra ahorro económico. El estado operacional de la cartera sigue siendo pending_new_execution.

2. Presentación técnica

2.1 Algoritmos genéticos

Un algoritmo genético es una metaheurística poblacional inspirada en la evolución biológica. Cada individuo representa una solución potencial; una función de aptitud cuantifica su calidad; la selección favorece individuos aptos; el cruzamiento combina información de dos padres; y la mutación introduce variabilidad. La repetición generacional permite explorar un espacio combinatorio sin enumerar exhaustivamente todas las alternativas.

2.2 Correspondencia biológica y computacional

Concepto evolutivo	Implementación en el proyecto
Individuo	Una cartera candidata de validación.
Cromosoma	Vector binario de longitud igual al número de candidatos.
Gen	Decisión de incluir (1) o excluir (0) una propuesta upgrade.
Fitness	Cobertura, diversidad, calidad histórica y concentración.
Selección	Torneo de cuatro individuos.
Cruzamiento	Combinación uniforme de genes de dos padres.
Mutación	Inversión aleatoria de bits.
Elitismo	Conservación de las cuatro mejores carteras.
Reparación	Ajuste para respetar exactamente el presupuesto de 20 casos.

2.3 Pertinencia de la técnica

Con 113 candidatos factibles y un presupuesto de 20 casos existen más de 10^{21} combinaciones posibles. La búsqueda exhaustiva resulta innecesaria y costosa. Además, la solución debe equilibrar varios objetivos que compiten entre sí: elegir solo los casos de mayor calidad reduce diversidad, mientras que maximizar solo diversidad puede incorporar casos menos sólidos. Este equilibrio convierte al problema en un candidato natural para algoritmos evolutivos.

3. Descripción formal del problema

3.1 Entradas

- layered_recommendations.csv, generado por pde evaluate-complete.
- resultadoPruebasGatling.txt, histórico corporativo de ancho fijo.
- Presupuesto experimental B=20.
- Parámetros genéticos, semilla y pesos auditables.

3.2 Variable de decisión

Para cada candidato i se define $x_i \in \{0,1\}$. Si $x_i=1$, el caso se incluye en la cartera; de lo contrario queda para una campaña posterior. La restricción presupuestaria exige $\sum x_i=20$.

3.3 Restricciones duras de seguridad

Campo	Condición obligatoria	Justificación
Estado	Success	Una ejecución fallida no puede respaldar un upgrade.
Performance	1	El caso debe aplicar a pruebas de rendimiento.
errorCount	0	No se evoluciona carga con errores observados.
p95	$0 < p95 \leq 1.500 \text{ ms}$	Se exige evidencia válida y headroom consistente con la política del Capstone.
Acción	upgrade	Solo se priorizan propuestas de evolución.
Estado online	pending_new_execution	La selección no reemplaza la reejecución.

Decisión de diseño: la seguridad se implementa como factibilidad previa. No es un peso del fitness; por tanto, un caso fallido nunca puede compensar su riesgo aportando diversidad.

4. Metodología y diseño del algoritmo

4.1 Preparación de los datos

6. Filtrar action=upgrade y online_validation_status=pending_new_execution.
7. Unir la propuesta con evidencia histórica mediante Build_Id.
8. Aplicar las restricciones duras de seguridad.
9. Eliminar duplicados por build, cuadrante propuesto y configuración.
10. Asignar un índice estable a cada candidato factible.

4.2 Representación cromosómica

El cromosoma es un vector binario de 113 posiciones. Ejemplo simplificado: [0,1,0,0,1,...]. Cada 1 representa un caso seleccionado. Después del cruzamiento y la mutación, una función de reparación agrega o elimina genes activos hasta recuperar exactamente 20 casos.

4.3 Función de fitness

Componente	Peso	Interpretación
Cobertura de pilares	0,30	Proporción de pilares representados.
Cobertura de componentes	0,20	Diversidad de componentes dentro del presupuesto.
Cobertura de cuadrantes	0,15	Representación de intensidades operacionales.
Calidad histórica	0,20	Combinación de estado, errores y margen de p95.
Diversidad de configuración	0,10	Variedad de concurrencia, iteraciones y tiempo esperado.
Cobertura de builds	0,05	Premia la representación de builds distintos.
Penalización de concentración	-0,10	Castiga la máxima cantidad de casos que comparten un build.

Los pesos son supuestos académicos explícitos. No representan costos económicos demostrados y pueden modificarse para realizar análisis de sensibilidad.

4.4 Parámetros

Parámetro	Valor
Presupuesto	20
Población	80
Generaciones	120
Probabilidad de cruzamiento	0.85
Probabilidad de mutación	0.03
Elite	4
Tamaño de torneo	4

Parámetro	Valor
Ejecuciones independientes	10
Semillas	42 a 51

4.5 Pseudocódigo

leer recomendaciones e histórico
filtrar upgrades pendientes y aplicar restricciones duras
eliminar duplicados y crear candidatos
crear población aleatoria con B genes activos

para cada generación:

- evaluar cobertura, diversidad, calidad y concentración
- conservar elite
- seleccionar padres por torneo
- aplicar cruzamiento uniforme
- mutar bits
- reparar cromosoma para mantener B casos

retornar mejor cartera y comparar con baselines

5. Implementación

La solución fue implementada en Python 3.11 sin dependencias externas para el núcleo evolutivo. El proyecto contiene módulos separados para lectura y depuración de datos, representación de candidatos, operadores genéticos, interfaz de línea de comandos, pruebas automáticas y generación de artefactos.

5.1 Estructura

Elemento	Responsabilidad
validation_portfolio/data.py	Lee el ancho fijo, integra Build_Id, filtra seguridad y elimina duplicados.
validation_portfolio/genetic.py	Define cromosoma, fitness, reparación, selección, cruzamiento, mutación y elitismo.
validation_portfolio/cli.py	Ejecuta 10 semillas y genera CSV, JSON y SVG.
tests/test_optimizer.py	Comprueba presupuesto, validez de cartera y funcionamiento generacional.
results/	Conserva solución, comparación, historial, corridas y cartera priorizada.

5.2 Reproducibilidad

La semilla base, parámetros, pesos y restricciones se guardan en methodology.json. Cada ejecución independiente registra su semilla y fitness. La cartera conserva Build_Id, pilar, componente, cuadrante, configuración propuesta, estado, Performance, errores, p95 y necesidad de aprobación humana.

5.3 Baselines

El algoritmo se compara con: (a) el fitness promedio de 100 carteras aleatorias válidas y (b) una heurística greedy que elige los 20 casos de mayor calidad individual. El baseline aleatorio mide cuánto aporta la búsqueda; el greedy evidencia el costo de ignorar diversidad y cobertura.

6. Resultados

6.1 Depuración y solución

Indicador	Resultado
Propuestas upgrade iniciales	275
Candidatos únicos y factibles	113
Presupuesto	20
Mejor fitness	0.99048667
Calidad histórica promedio	0.952433
Penalización de concentración	0.0
Estado online	pending_new_execution

6.2 Comparación

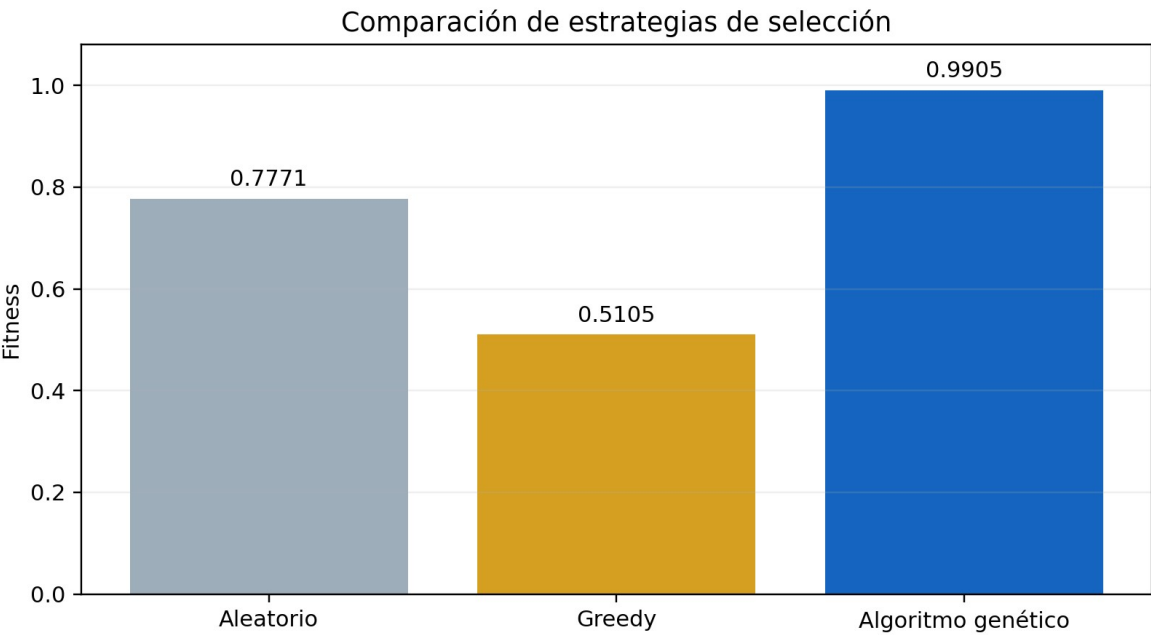


Figura 1. Comparación del fitness entre estrategias de selección.

Método	Fitness	Lectura
Greedy por calidad	0,51053	Alta calidad individual, baja diversidad.
Promedio aleatorio (100)	0,77711	Cobertura moderada sin búsqueda dirigida.
Algoritmo genético	0,99049	Mejor equilibrio entre todos los objetivos.

6.3 Convergencia

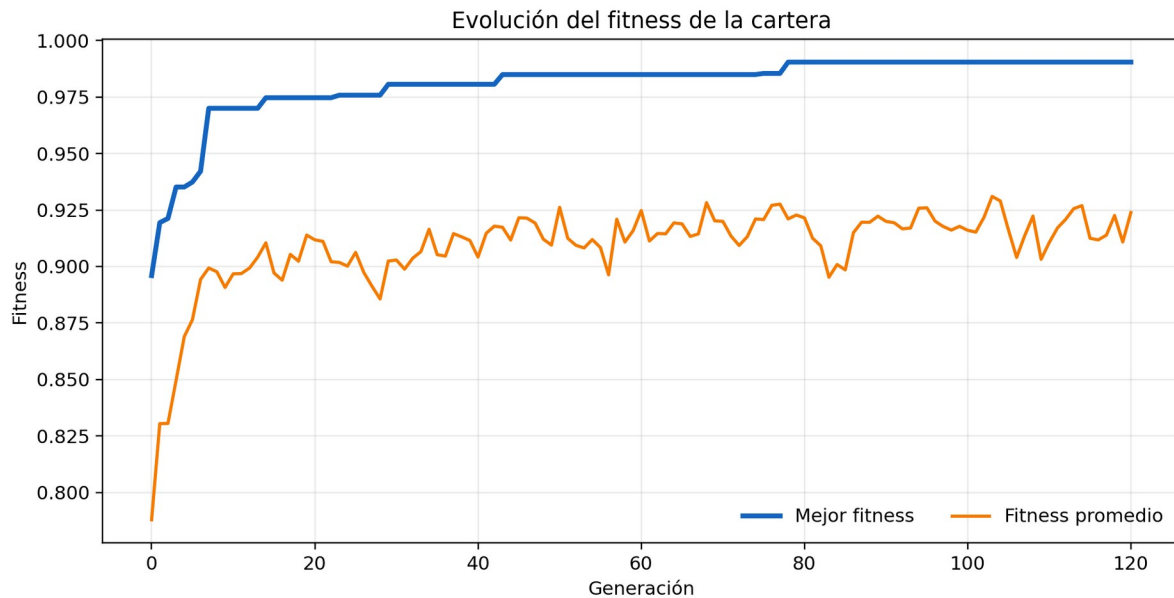


Figura 2. Evolución del mejor fitness y del fitness promedio durante 120 generaciones.

El mejor fitness aumentó de 0,89595 a 0,99049. Se alcanzó el 95 % del resultado final en la generación 6, el 99 % en la generación 29 y el 99,9 % en la generación 78. La línea promedio oscila porque el cruzamiento y la mutación mantienen diversidad, mientras el elitismo impide perder la mejor solución.

6.4 Estabilidad

Corrida	Semilla	Fitness
1	42	0.985750
2	43	0.980303
3	44	0.984897
4	45	0.985313
5	46	0.980652
6	47	0.980553
7	48	0.985338
8	49	0.985066
9	50	0.985668
10	51	0.990487

Las diez ejecuciones obtuvieron promedio 0,98440, desviación estándar poblacional 0,00298, mínimo 0,98030 y máximo 0,99049. La baja dispersión muestra estabilidad frente al componente aleatorio.

6.5 Cobertura y seguridad

Dimensión	Resultado
Builds diferentes	20
Pilares	4
Componentes	12
Cuadrantes	7
Configuraciones diferentes	20
Estado Success	20/20
Performance=1	20/20
errorCount=0	20/20
Rango p95	386-1.125 ms

7. Discusión

7.1 Interpretación

El algoritmo genético supera ampliamente a ambos comparadores porque optimiza la cartera como conjunto. El greedy privilegia casos individualmente favorables, pero repite contextos y obtiene bajo fitness. La selección aleatoria logra diversidad parcial, aunque no coordina sistemáticamente cobertura, calidad y concentración. La búsqueda evolutiva encuentra una combinación con cobertura normalizada completa en las dimensiones definidas y calidad histórica promedio de 0,95243.

7.2 Valor para el Capstone

El aporte se ubica después de las cuatro capas existentes. No duplica Random Forest, análisis de umbrales, GroupKFold, reglas review/maintain/upgrade ni perfiles de pares. Resuelve una pregunta operacional nueva: ante numerosos upgrades pendientes, ¿qué conjunto conviene validar primero para producir evidencia representativa con un presupuesto acotado?

7.3 Seguridad y corrección metodológica

Una versión preliminar trataba estado y errores como componentes penalizados del fitness. Ese diseño permitía que un caso fallido compensara su riesgo mediante diversidad. La versión final corrige el problema transformando Estado=Success, Performance=1, errorCount=0 y p95 válido en restricciones de factibilidad. Además, la función v2 reemplaza la penalización 1-cobertura de builds, que era colineal con el premio de cobertura, por la máxima concentración en un mismo build, una señal independiente. Ambas correcciones separan seguridad, cobertura y concentración, evitando doble contabilización.

7.4 Limitaciones

- La calidad histórica es un proxy; no garantiza el resultado futuro.
- Los pesos del fitness son supuestos académicos y requieren análisis con especialistas.
- La unión por Build_Id puede agrupar varios escenarios del mismo build; una identidad de ejecución más granular mejoraría trazabilidad.
- El fitness combina objetivos normalizados y no equivale a ahorro monetario ni probabilidad calibrada de éxito.
- No existe validación online hasta ejecutar Gatling con la configuración aprobada.
- Los resultados provienen de un único histórico organizacional y no demuestran generalización externa.

7.5 Amenazas a la validez

Tipo	Amenaza	Mitigación
Interna	Dependencia de Build_Id para integrar evidencia.	Mantener trazabilidad y revisar manualmente la cartera.
Constructo	El fitness representa preferencias diseñadas.	Pesos explícitos, baselines y análisis de sensibilidad futuro.
Externa	Datos de una sola organización.	Validar en nuevos microservicios y periodos.

Tipo	Amenaza	Mitigación
Conclusión	El alto fitness puede confundirse con éxito online.	Conservar pending_new_execution y aprobación humana.

8. Conclusiones y proyecciones

8.1 Conclusiones

Se implementó exitosamente un algoritmo genético para seleccionar una cartera de validación bajo presupuesto. La representación binaria, los operadores evolutivos, la reparación y la función multiobjetivo permitieron explorar un espacio combinatorio muy grande y obtener una solución estable y reproducible.

La cartera final contiene 20 casos seguros según las restricciones históricas, representa 20 builds, 4 pilares, 12 componentes, 7 cuadrantes y 20 configuraciones, y presenta concentración nula por build. El algoritmo obtuvo fitness 0,99049, superior al promedio aleatorio y al greedy. Esto demuestra la utilidad de los algoritmos genéticos para planificación experimental dentro del contexto real del Capstone.

Conclusión operacional: la cartera está lista para revisión humana y diseño de campaña; todavía no está validada como upgrade. La evidencia definitiva requiere ejecutar Gatling y comprobar el contrato de éxito del Capstone.

8.2 Proyecciones

11. Ejecutar una primera muestra aprobada y realimentar el fitness con resultados online.
12. Incorporar costo estimado, duración e infraestructura de cada prueba como restricciones de presupuesto.
13. Realizar análisis de sensibilidad de pesos y parámetros genéticos.
14. Comparar con NSGA-II para obtener un frente de Pareto sin agregar objetivos en un único fitness.
15. Usar una identidad granular de escenario o endpoint, además de Build_Id.
16. Integrar el optimizador como comando experimental de la CLI pde, manteniendo aprobación humana.

9. Referencias

- Breiman, L. (2001). Random forests. *Machine Learning*, 45, 5-32.
- Deb, K. (2001). *Multi-Objective Optimization Using Evolutionary Algorithms*. Wiley.
- Eiben, A. E., & Smith, J. E. (2015). *Introduction to Evolutionary Computing* (2nd ed.). Springer.
- Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.
- Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
- Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- Equipo Capstone Grupo 8. (2026). *Gatling AI Performance Copilot v1.6.0: documentación técnica y pipeline histórico*. Repositorio del proyecto.
- Contreras, R. (2026). *Material de clases MIA1 y MIA2: algoritmos genéticos simples y avanzados*. Universidad Adolfo Ibáñez.

Anexo A. Cartera seleccionada

La tabla conserva los identificadores y parámetros necesarios para planificar la campaña. Todos los registros requieren aprobación humana.

Pr.	Build	Pilar	Componente	Q	Concurr.	Iter.	Resp.	p95
1	447155	Loyalty	Loyalty-Shelby	7	very_low	medium	very_high	386
2	462262	Loyalty	Loyalty-SuperDruperKit	9	high	medium	very_high	391
3	454448	Loyalty	Loyalty-SuperDruperKit	9	high	high	medium	394
4	447585	Loyalty	Loyalty-Shelby	6	high	medium	medium	399
5	483176	Loyalty	Loyalty-TheLastOfPlus	7	very_low	low	very_high	406
6	471585	Loyalty	Loyalty-CommunityTeam	7	low	medium	very_high	411
7	484266	Loyalty	Loyalty-CommunityTeam	9	high	high	high	413
8	447643	Loyalty	Loyalty-TheLastOfPlus	7	low	high	medium	418
9	443266	Loyalty	Loyalty-Shelby	1	very_low	low	low	450
10	442366	COE	COE	5	medium	medium	medium	501
11	495876	COE	COE	9	very_high	high	medium	538
12	467724	Loyalty	Loyalty-TheLastOfPlus	4	low	low	medium	589
13	485004	Loyalty	Loyalty-CommunityTeam	9	high	medium	high	594
14	494047	OSS	OSS-LosBorbotones	8	medium	medium	high	676
15	477013	OSS	OSS-Shazam	7	low	low	very_high	677
16	468337	OSS	OSS-Shazam	9	high	high	low	686
17	478133	OpenBanking	OpenBanking-HubDeDatos	5	medium	low	medium	751
18	479170	Loyalty	Loyalty-TheOffice	4	very_low	low	medium	776
19	447840	Loyalty	Loyalty-TheOffice	7	very_low	low	high	835
20	491945	Loyalty	Loyalty-CommunityTeam	7	very_low	medium	high	1125

Nota: Pr.=prioridad; Q=cuadrante propuesto; p95 expresado en milisegundos. Estado de todos los casos: pending_new_execution.

Anexo B. Correspondencia con la rúbrica

Criterio	Ubicación	Evidencia
Introducción (0,5)	Secciones 1.1-1.5	Antecedentes, justificación, objetivos y alcance.
Presentación técnica (0,5)	Sección 2	Explicación resumida de AG y correspondencia computacional.
Descripción del problema (0,5)	Sección 3	Variables, restricciones y formulación formal.
Algoritmo (0,5)	Sección 4	Cromosoma, fitness, operadores, parámetros y pseudocódigo.
Resultados (1,0)	Sección 6 y Anexo A	Resultados coherentes, gráficos, comparación y cartera.
Discusión (1,5)	Sección 7	Comparaciones, interpretación, limitaciones y validez.
Conclusiones (1,0)	Sección 8	Respuesta al objetivo, alcance y proyecciones.
Referencias (0,5)	Sección 9	Fuentes fundamentales, técnicas y del curso.